

Adaptive Collaborative Air-Ground Edge Caching in Dynamic Vehicular Network Scenarios

Fei Luo[†]*, Junhua Wang^{†‡}

[†]College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China

[‡]State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

[†]luofei10@nuaa.edu.cn, [‡]jhual207@nuaa.edu.cn

*Corresponding author

Abstract—Mobile edge caching is a promising technique for supporting the applications of Intelligent Transportation Systems (ITS), which involve huge volumes of image and video contents. Specifically, critical challenges arise in meeting the contents requirements of moving vehicles. In this work, we investigate the collaborative contents delivery in air-ground integrated vehicular networks, where unmanned aerial vehicles (UAVs) and roadside units (RSUs) cache popular contents and further deliver it to nearby vehicles. Efficient contents placement is crucial because of the high mobility of vehicles and UAVs, and the limited caching capacity of edge nodes. To address this, we formulate the heterogeneous cache optimization problem as a generalized multi-dimensional multiple knapsack problem (MMKP), which is NP-hard. In particular, we propose an adaptive collaborative caching optimization (ACCO) algorithm that employs the Kernel Search (KS) framework to find a near-optimal caching scheme, with dynamically adjustable dimensions and solution time for subproblems. Furthermore, we use the k-means clustering algorithm to categorize contents based on service latency, caching cost, and contents size, adding a greedy-based load balancing strategy to achieve cost-effective optimal matching of contents and edge nodes. Comprehensive simulations verify the effectiveness of the proposed cooperative edge caching architecture and the ACCO algorithm in reducing service delay and caching costs for contents delivery.

Keywords—Cooperative edge caching; air-ground integrated vehicular networks; cache placement; kernel search; server load balancing;

I. INTRODUCTION

In recent years, the automotive industry has been focusing more on in-car entertainment systems that heavily rely on real-time video files. These files need to be delivered quickly and without interruption for a better overall experience. Moreover, this puts a lot of pressure on vehicular networks that only use cellular networks for data transmission [1].

Fortunately, mobile edge caching is crucial for 6G networks as it reduces latency and addresses backhaul burden. It is becoming more cost-efficient with declining storage costs [2]. Nevertheless, due to the unique features of vehicular networks, such as rapidly changing network topology and high vehicle speeds, it is difficult to rely on only one wireless communication technology to support Intelligent Transportation Systems (ITS) services. To address this issue, heterogeneous vehicular networks (H-VNets) present

a promising solution. By allowing base stations (BSs) and roadside units (RSUs) to work together to cache contents, vehicular communication costs can be reduced [3]. Unmanned aerial vehicles (UAVs) are gaining interest due to their low cost, flexibility, and mobility. They can act as edge servers in certain scenarios to meet user demands. For instance, in [4], UAVs and ground BSs cached videos during off-peak hours to serve mobile users during peak hours, compensating for the limited coverage of surface infrastructure. Additionally, future ITS plan to implement "cinemas on wheels" applications, allowing mobile users to access multimedia contents during heavy traffic [5]. This highlights the importance of developing new applications supported by edge cache servers. Thus, cooperative edge caching in H-VNets is necessary to reduce network congestion and improve the driving experience of users. Previous studies have studied caching optimization in mixed networks but have not included UAVs, RSUs, and BSs working together to deliver contents for vehicles in vehicular networks. Additionally, optimizing service latency and caching costs simultaneously is an unexplored area of mobile edge caching.

Faced with the aforementioned challenges, we propose an air-ground cooperative edge caching architecture that allows UAVs, RSUs, and BSs to collaborate in caching and delivering popular contents to moving vehicles in dynamic vehicular networks. Specifically, we formulate a joint optimization problem of UAVs and RSUs cache placement as a generalized multi-dimensional multiple knapsack problem (MMKP) to minimize service delay by enabling seamless contents delivery and reduce caching costs by storing contents on low-cost servers. This problem is inherently NP-hard. To tackle this challenge, we propose an adaptive collaborative caching optimization (ACCO) algorithm that utilizes the Kernel Search (KS) framework to solve subproblems with dynamically adjustable dimensions and solution time. Additionally, we use the k-means clustering algorithm to categorize contents based on service latency, caching cost, and contents size, and add a greedy-based load balancing strategy to effectively match contents with edge nodes. Simulation results demonstrate the effectiveness of the proposed caching architecture and algorithm in minimizing

service delay and caching costs compared to other classical algorithms.

II. SYSTEM MODEL AND PROBLEM FORMULATION

A. Scenario Description and Assumptions

Fig. 1 shows the proposed air-ground cooperative edge caching scenarios with U UAVs, R RSUs, B BSs, and V vehicles. Let $\mathcal{U} = \{1, 2, \dots, U\}$, $\mathcal{R} = \{1, 2, \dots, R\}$, $\mathcal{B} = \{1, 2, \dots, B\}$, and $\mathcal{V} = \{1, 2, \dots, V\}$ denote the set of UAVs, RSUs, BSs, and vehicles, respectively. A vehicle $v \in \mathcal{V}$ driving within this area may request popular contents, such as news and video services. Each UAV $u \in \mathcal{U}$ and RSU $r \in \mathcal{R}$ is connected to an edge caching server, which caches limited popular contents for nearby vehicles. Additionally, the BS $b \in \mathcal{B}$ is a cloud server that contains all requested contents and maintains powerful communication capabilities. $\mathcal{F} = \{1, 2, \dots, F\}$ represents the index set of available contents. The size of contents $f \in \mathcal{F}$ is z_f , and the contents size vector is $\mathcal{Z} = [z_1, z_2, \dots, z_f, \dots, z_F]$. Moreover, the timeline is separated into T time slots of equal length, indexed by t .

The UAV, RSU, and BS use cellular vehicle-to-everything (C-V2X) interfaces to communicate with vehicles. While RSUs and BSs have higher contents caching costs due to deployment and maintenance expenses, UAVs have low transmission rates due to interference. In edge cache applications, vehicular users may switch between different servers for seamless communication. Our focus is on collaborative contents delivery in vehicular networks to meet both delay and caching cost requirements. Different spectrum frequencies are used for UAV-to-vehicle (U2V), RSU-to-vehicle (R2V), and BS-to-vehicle (B2V) communications to avoid co-channel interference.

B. Channel Model

In this section, we describe the channel models between vehicles and various types of edge nodes.

1) *U2V Channel Model*: Wireless signals between UAVs and vehicles can be weakened by reflections and diffractions. To address this, we use the path-loss model from [6]. This model considers both line-of-sight (LoS) and non-line-of-sight (NLoS) channels. The expressions are given as follows.

$$L_{LoS} = \eta_{LoS} (4\pi f_{u,v}/c)^2 d_{u,v}^2, \quad (1)$$

$$L_{NLoS} = \eta_{NLoS} (4\pi f_{u,v}/c)^2 d_{u,v}^2, \quad (2)$$

where $f_{u,v}$ is the carrier frequency and $d_{u,v}$ is the distance between UAV u and vehicle v . η_{LoS} and η_{NLoS} are shadowing and diffraction losses for LoS and NLoS, respectively. The probability of LoS and NLoS links between a UAV and a vehicle depends on their elevation angle and environment, which can be expressed as

$$p_{LoS} = \frac{1}{1 + a \exp(-b(\theta - a))}, \quad (3)$$

$$p_{NLoS} = 1 - p_{LoS}, \quad (4)$$

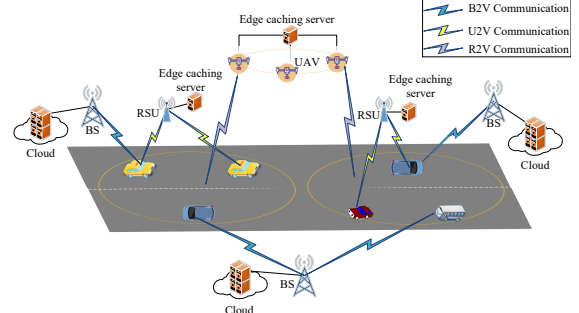


Fig. 1. Collaborative air-ground edge caching in dynamic vehicular scenarios.

where a and b are environmental constants, and θ is the elevation angle in radians. The average path loss for LoS and NLoS can be calculated accordingly.

$$\bar{L} = p_{LoS} L_{LoS} + p_{NLoS} L_{NLoS}. \quad (5)$$

Then the transmission rate between UAV u and vehicle v is given by

$$R_{u,v} = B_{u,v} \log_2 \left(1 + \frac{P_{u,v}}{\bar{L} \sigma_u^2 B_{u,v}} \right), \quad (6)$$

where $B_{u,v}$ is the bandwidth between UAV u and vehicle v , and $P_{u,v}$ is the transmission power. σ_u^2 represents the noise power.

2) *R2V Channel Model*: Radio signals are scattered in urban areas with dense buildings. The fading channel is assumed to follow the Rician distribution, and the channel gain is given by

$$h_{r,v} = \vartheta \cdot A_d \left(\frac{c}{4\pi f_{r,v} d_{r,v}} \right)^{d_e}, \quad (7)$$

where ϑ is a constant. A_d is the antenna gain, and d_e is the path loss exponent. $f_{r,v}$ is the carrier frequency, and $d_{r,v}$ is the distance between RSU r and vehicle v .

The transmission rate from RSU r to vehicle v can be expressed as

$$R_{r,v} = B_{r,v} \log_2 \left(1 + \frac{P_{r,v} h_{r,v}}{\sigma_r^2} \right), \quad (8)$$

where $B_{r,v}$ and $P_{r,v}$ represent the communication bandwidth and transmit power between the RSU and vehicles, respectively. σ_r^2 represents the receiver noise power, including potential interference and thermal noise.

3) *B2V Channel Model*: We consider the small-scale fading effect in B2V communication and compute the channel gain as follows.

$$h_{b,v} = g_0 \frac{d_0}{d_{b,v}^4}, \quad (9)$$

where g_0 and d_0 are the path-loss constant and reference distance, respectively, and $d_{b,v}$ is the distance between the base station and the vehicle.

The achievable rate of delivering the contents from BS b to vehicle v is formulated as

$$R_{b,v} = B_{b,v} \log_2 \left(1 + \frac{P_{b,v} h_{b,v}}{\sigma_b^2} \right), \quad (10)$$

where $B_{b,v}$ and $P_{b,v}$ are the communication bandwidth and transmit power between the BS and vehicles, respectively. σ_b^2 denotes the noise power at the receiver.

C. Caching Model

According to the Zipf distribution, we can assume that file popularity follows a certain pattern. Therefore, we can

express the popularity of f -ranked contents as follows.

$$P(f) = \frac{\frac{1}{\xi^j}}{\sum_{f=1}^F \frac{1}{f^j}}, \quad (11)$$

where j represents the Zipf exponent indicating the skewness of popularity, and ξ denotes the ranking of contents request frequency. Cache placement vectors $\mathbb{A} = \alpha_{u,f,t}$ and $\mathbb{B} = \beta_{r,f,t}$ represent UAV u and RSU r for each contents and time slot, where $\alpha_{u,f,t}, \beta_{r,f,t} \in [0, 1]$. Cached contents in each UAV and RSU should not exceed their respective available caching capacities Q_u and Q_r .

D. Problem Formulation

With the above analysis, the service delay is given by

$$St = \sum_{u=1}^U \sum_{f=1}^F \alpha_{u,f,t} \cdot \frac{z_f}{R_{u,v}} + \sum_{r=1}^R \sum_{f=1}^F \beta_{r,f,t} \cdot \frac{z_f}{R_{r,v}} + \left(1 - \sum_{u=1}^U \sum_{f=1}^F \alpha_{u,f,t} - \sum_{r=1}^R \sum_{f=1}^F \beta_{r,f,t} \right) \cdot \frac{z_f}{R_{b,v}}. \quad (12)$$

The caching costs is described as

$$Sc = \rho_u N_u S_u \cdot \sum_{u=1}^U \sum_{f=1}^F \alpha_{u,f,t} + \rho_r N_r S_r \cdot \sum_{r=1}^R \sum_{f=1}^F \beta_{r,f,t} + \rho_b N_b S_b \cdot \left(F - \sum_{u=1}^U \sum_{f=1}^F \alpha_{u,f,t} - \sum_{r=1}^R \sum_{f=1}^F \beta_{r,f,t} \right). \quad (13)$$

Here, ρ_u , ρ_r , and ρ_b are the costs of renting cache resources from UAVs, RSUs, and BSs, respectively. N_u , N_r , and N_b are the number of edge nodes per unit area, and S_u , S_r , and S_b are the number of popular contents cached.

We formulate a joint optimization problem for caching resource allocation between UAVs and RSUs to make a trade-off between the two measures of service delay and caching costs. The optimization problem is as follows:

$$\begin{aligned} \text{P1: } \min_{\mathbb{A}, \mathbb{B}} & \sum_{t=1}^T \epsilon \cdot St + (1 - \epsilon) \cdot Sc \\ \text{s.t. } & (a) \quad \alpha_{u,f,t}, \beta_{r,f,t} \in \{0, 1\}, \\ & (b) \quad \sum_{f=1}^F \alpha_{u,f,t} + \beta_{r,f,t} \leq 1, \\ & (c) \quad \sum_{f=1}^F z_f \cdot \alpha_{u,f,t} \leq Q_u, \sum_{f=1}^F z_f \cdot \beta_{r,f,t} \leq Q_r, \\ & (d) \quad \sum_{f=1}^F \frac{z_f}{R_{u,v}} \cdot \alpha_{u,f,t} \leq Lu, \sum_{f=1}^F \frac{z_f}{R_{r,v}} \cdot \beta_{r,f,t} \leq Lr, \\ & (e) \quad u \in \mathcal{U}, r \in \mathcal{R}, b \in \mathcal{B}, t \in \mathcal{T}, f \in \mathcal{F}, \end{aligned} \quad (14)$$

where $\epsilon \in [0, 1]$ is a weighting parameter. Constraint (14b) puts a limit on the maximum number of contents that can be cached in UAVs and RSUs, while constraint (14c) governs the maximum allowable connection time between UAVs, RSUs, and vehicles. Unlike the traditional knapsack problem, the proposed problem P1 combines multiple dimensions (i.e., transmission delay and caching capability) and multiple

knapsacks (i.e., multiple edge nodes). Although previous studies have suggested exact algorithms (e.g., branch and bound, dynamic programming) to solve the problem, brute force methods are too costly for larger problem sizes. Inspired by the KS framework, we propose the ACCO algorithm for large-scale MMKP.

III. PROPOSED ALGORITHM

KS is a heuristic framework that solves multi-dimensional knapsack and portfolio selection problems by identifying kernel variables likely to be in the optimal solution and solving subproblems with those variables and additional ones [7]. To improve the performance of the KS framework, we propose the ACCO algorithm, which uses unsupervised learning and a greedy strategy. Here are the main steps:

- 1) **The initialization phase:** We relax the linear programming of the original problem and use k-means clustering algorithm to obtain an upper bound.
- 2) **The iterative solving process:** The MMKP is decomposed into smaller subproblems and solved iteratively using CPLEX tool for feasible solutions.
- 3) **The dynamic adjustment phase:** We dynamically adjust the dimensions and solution time of subproblems. Specifically, we optimize the solution results further by using a greedy-based load balancing strategy.

Algorithm 1 and Fig. 2 present the pseudo-code and process of the ACCO algorithm, respectively. In the following, we describe the detailed algorithm procedure.

A. The initialization phase

A good upper bound is essential to prune the infeasible search space and achieve fast convergence. As shown in P2, adjusting constraint (14a) in the objective function gives us the Linear Programming relaxation of P1.

$$\begin{aligned} \text{P2: } \min_{\mathbb{A}, \mathbb{B}} & \sum_{t=1}^T \epsilon \cdot St + (1 - \epsilon) \cdot Sc \\ \text{s.t. } & (a) \quad 0 \leq \alpha_{u,f,t}, \beta_{r,f,t} \leq 1, \\ & (14b) \sim (14e). \end{aligned} \quad (15)$$

We first solve P2 to obtain the initial kernel set Λ by selecting variables with positive values in the continuous relaxation process. This gives us the upper bound z^{LP} . Then, the infeasible solution z^{LP} is converted into a feasible one using the K -means clustering algorithm, which is a popular prototype-based clustering algorithm. It partitions the $|F|$ contents into $|K|$ groups (i.e., the number of types of edge nodes), where $|K| \leq |F|$. Each contents belongs to the cluster with the nearest mean (cluster centroid), which serves as a prototype of the cluster. Different cluster centroids have the farthest distance from each other. We provide characteristic data $(\mathbf{x}_1, \dots, \mathbf{x}_F)$ for contents (i.e., the service delay, caching costs, and contents size) and sets $\mathcal{S} \triangleq \{s_1, \dots, s_K\}$. The K -means clustering algorithm

minimizes the within-cluster sum of squares, i.e., squared Euclidean distances. Formally, the objective is to find

$$\arg \min_S \sum_{k=1}^K \sum_{\mathbf{x}_f \in s_k} \|\mathbf{x}_f - \mu_k\|^2, \quad (16)$$

where μ_k is the mean position of users in cluster s_k .

B. The iterative solving process

In this phase, we use the KS framework to traverse buckets of subproblems and obtain a feasible solution. Firstly, variables not in the kernel set are divided into bucket sequences BK_i , generating NB buckets. This decomposition transforms a large-scale integer linear programming (ILP) problem into smaller subproblems. Therefore, P2 can be transformed into:

$$\begin{aligned} \text{P3: } \min_{\mathbf{A}, \mathbf{B}} \sum_{t=1}^T \sum_{n=1}^{NB} \epsilon \cdot \tilde{S}t + (1 - \epsilon) \cdot \tilde{S}c \\ \text{s.t. } (a) \quad \sum_{f=1}^m z_f \cdot \alpha_{u,f,t} \leq \tilde{Q}u, \quad \sum_{f=1}^m z_f \cdot \beta_{r,f,t} \leq \tilde{Q}r, \\ (b) \quad \sum_{f=1}^m \frac{z_f}{R_{u,v}} \cdot \alpha_{u,f,t} \leq \tilde{L}u, \quad \sum_{f=1}^m \frac{z_f}{R_{r,v}} \cdot \beta_{r,f,t} \leq \tilde{L}r, \end{aligned} \quad (17)$$

Here, m represents the number of contents in each subproblem, and will be adjusted dynamically later. $\tilde{Q}u$, $\tilde{Q}r$, $\tilde{L}u$, and $\tilde{L}r$ are fixed values. $\tilde{S}t$ and $\tilde{S}c$ can be expressed as follows:

$$\begin{aligned} \tilde{S}t = \sum_{u=1}^U \sum_{f=1}^m \alpha_{u,f,t} \cdot \frac{z_f}{R_{u,v}} + \sum_{r=1}^R \sum_{f=1}^m \beta_{r,f,t} \cdot \frac{z_f}{R_{r,v}} \\ + \left(1 - \sum_{u=1}^U \sum_{f=1}^m \alpha_{u,f,t} - \sum_{r=1}^R \sum_{f=1}^m \beta_{r,f,t} \right) \cdot \frac{z_f}{R_{b,v}}, \end{aligned} \quad (18)$$

$$\begin{aligned} \tilde{S}c = \rho_u N_u S_u \cdot \sum_{u=1}^U \sum_{f=1}^m \alpha_{u,f,t} + \rho_r N_r S_r \cdot \sum_{r=1}^R \sum_{f=1}^m \beta_{r,f,t} \\ + \rho_b N_b S_b \cdot \left(m - \sum_{u=1}^U \sum_{f=1}^m \alpha_{u,f,t} - \sum_{r=1}^R \sum_{f=1}^m \beta_{r,f,t} \right). \end{aligned} \quad (19)$$

We solve the set $\Lambda \cup BK_i$ (i.e., the current bucket) using the CPLEX tool by adding the following two constraints.

$$\sum_{f \in BK_i} \alpha_{u,f,t} + \beta_{r,f,t} > 1, \quad z_{cut} = z^* + 1, \quad (20)$$

where z_{cut} is the cut-off value for the objective function of the problem. Secondly, we label the status based on the quality and time of the subproblem solution to evaluate instance difficulty. The status is *feasible* if a feasible solution is found. If no feasible solution is found before the time limit, the status is *timeLimit*. If the subproblem results are infeasible due to the cutoff value, the status is *cutoff*. Finally, if a subproblem solution approaches optimality before reaching the time limit, the status is *optimal*. x^* represents the best feasible solution when the status is *feasible* or *optimal*, and its value is z^* .

Algorithm 1 The adaptive collaborative caching optimization (ACCO) algorithm.

Input: (m, Δ, t_{op}) , $(m^{\text{slow}}, \Delta^{\text{slow}}, t^{\text{slow}})$, $(m^{\text{fast}}, \Delta^{\text{fast}}, t^{\text{fast}})$, percentage γ, ι , MMKP instance I
Output: the best feasible solution x^* and its value z^*

```

1:  $n_{sub}, n_{pos} \leftarrow 0, t_1^{\max} \leftarrow \gamma t_{max}$ 
2: Derive the upper bound and the initial kernel:
    $(x^{LP}, z^{LP}, \Lambda) \leftarrow \text{LP\&CLUSTER}(I)$ 
    $\Lambda \leftarrow \text{INITIALKERNEL}(x^{LP}, z^{LP})$ 
3:  $(x^*, z^*, t_{elapsed}, status) \leftarrow \text{SOLVEKERNEL}(I, \Lambda, \emptyset, t_{op})$ 
4: if  $status! = \text{timeLimit}$  then
5:    $n_{pos} \leftarrow n_{pos} + 1$ 
6: end if
7: if  $status = \text{feasible}$  or  $status = \text{optimal}$  then
8:    $(\Lambda) \leftarrow \text{GREEDYMATCH}(z^*, z^{LP}, x^{LP})$ 
9: end if
10:  $n_{sub} \leftarrow n_{sub} + 1$ 
11: while  $t_{elapsed} < t_{max}$  do
12:   for  $i = 1$  to  $NB$  do
13:      $BK_i \leftarrow \text{BUILDBUCKETS}(F \setminus \Lambda, m)$ 
14:      $t \leftarrow \min(t, t_{max} - t_{elapsed})$ 
15:     Solve the subproblem using the CPLEX tool:
      $(x^{ILP}, z^{ILP}, t_{used}, status) \leftarrow \text{SOLVEKERNEL}$ 
      $(I, \Lambda \cup BK_i, z_{cut}, t_{op})$ 
16:     if  $status! = \text{timeLimit}$  then
17:        $n_{pos} \leftarrow n_{pos} + 1$ 
18:     end if
19:      $n_{sub} \leftarrow n_{sub} + 1, t_{elapsed} \leftarrow t_{elapsed} + t_{used}$ 
20:     if  $status = \text{optimal}$  or  $status = \text{feasible}$  then
21:        $(x^*, z^*) \leftarrow (x^{ILP}, z^{ILP})$ 
22:       Select items to be included in the kernel:  $(\Lambda) \leftarrow$ 
        $\text{GREEDYMATCH}(z^*, z^{LP}, x^{LP})$ 
23:     end if
24:   end for
25:    $m \leftarrow m + \Delta$ 
26:   Adjust the dimensions of subproblems and solution time:
27:   if  $t_{elapsed} \geq t_1^{\max}$  then
28:     if  $\frac{n_{pos}}{n_{sub}} \geq \iota$  then  $(m, \Delta, t_{op}) \leftarrow (m^{\text{fast}}, \Delta^{\text{fast}}, t^{\text{fast}})$ 
29:   else  $(m, \Delta, t_{op}) \leftarrow (m^{\text{slow}}, \Delta^{\text{slow}}, t^{\text{slow}})$ 
30:   end if
31: end while
32: return  $(x^*, z^*)$ 

```

C. The dynamic adjustment phase

On this basis, we adjust subproblem dimensions and solution time based on instance information gathered during iterative solving. If less than ι of *timeLimit* subproblems time out, the instance is considered *fast*. Otherwise, it is *slow*. After each bucket iteration, we update the initial bucket size m , higher computing time limit t_{op} , and bucket size increase Δ based on whether the instance is fast or slow.

Last but not least, we design a greedy-based load balancing strategy for selecting items for the kernel. The strategy considers service latency, caching costs, and request frequency to optimize cache placement for UAVs and RSUs. To account for multiple requests of the same contents in the same area, we define value ratios $e_{f,u}$ and $e_{f,r}$ as

$$\begin{aligned} e_{f,u} = \varphi \cdot \left| \frac{\epsilon * \left(\frac{z_f}{R_{b,v}} - \frac{z_f}{R_{u,v}} \right)}{(1 - \epsilon) * (\rho_b \cdot N_b \cdot S_b - \rho_u \cdot N_u \cdot S_u)} \right| \\ + (1 - \varphi) \cdot W_f, \end{aligned} \quad (21)$$

$$e_{f,r} = \varphi \cdot \left| \frac{\epsilon * \left(\frac{z_f}{R_{b,v}} - \frac{z_f}{R_{r,v}} \right)}{(1 - \epsilon) * (\rho_b \cdot N_b \cdot S_b - \rho_r \cdot N_r \cdot S_r)} \right| + (1 - \varphi) \cdot W_f, \quad (22)$$

where $\varphi \in [0, 1]$ is a weighting parameter and W_f denotes the requested frequency of contents f . The BS is requested if both $e_{f,u}$ and $e_{f,r}$ are greater than 1. If both are less than or equal to 1, the values are compared. If $e_{f,u}$ is less than $e_{f,r}$, the contents is cached on the UAV. Otherwise, it is cached on the RSU. When the contents size is large, service delay has a greater impact than caching costs. However, frequently requested larger contents may be cached in UAV or RSU instead of the BS to improve efficiency. Thus, the ratio of $e_{f,u}$ and $e_{f,r}$ values can help the KS framework improve the performance of the algorithm.

IV. SIMULATION RESULTS

A. Simulation Setup

In this section, we will build the simulation model and implement the ACCO algorithm using Matlab. The simulation scenario includes 5 UAVs and 5 RSUs. UAVs fly along a fixed trajectory with a radius ϖ and height h , which are set to 400 m and 100 m, respectively. Each RSU antenna covers an area of 500 m around it. The size of each contents is within the range of [0, 50] MB. The size of each contents is within the range of [0, 50] MB. In simulations, the transmitted power of UAVs $P_{u,v}$, RSUs $P_{r,v}$, and BS $P_{b,v}$ is set to 0.2 W, 0.1 W, and 1 W, respectively. We consider the allocated bandwidth $B_{u,v}$, $B_{r,v}$, and $B_{b,v}$ to be 3 MHz, 1 MHz, and 2.5 MHz, respectively. The distance from the vehicle to the RSU is 35 m, and the distance to the BS is 30 m. Due to unpredictable hardware factors, the unit costs of renting caching resources from edge nodes are randomly set within a certain interval.

Comparison algorithms include the optimal solution (OPT) solved using the CPLEX tool, and four existing optimization algorithms which are shown as follows:

- **The CRS-PSO algorithm** [8] : It modifies the particle swarm optimization algorithm by adding cache replacement for faster convergence and better performance.
- **The ISA algorithm** [9] : It improves simulated annealing by extracting two kinds of solution spaces, resulting in a higher quality approximate solution.
- **The GABG algorithm** [10] : It combines the genetic algorithm with a greedy strategy to reduce solving time and improve accuracy.
- **The PBIG algorithm** [11] : It enhances the iterative greedy algorithm by generating an initial set of solutions using destruction and reconstruction phases.

B. Results

Fig. 3 shows the service delay and caching costs for six algorithms with different numbers of requests. As the number of requests increases, so do the service delay and

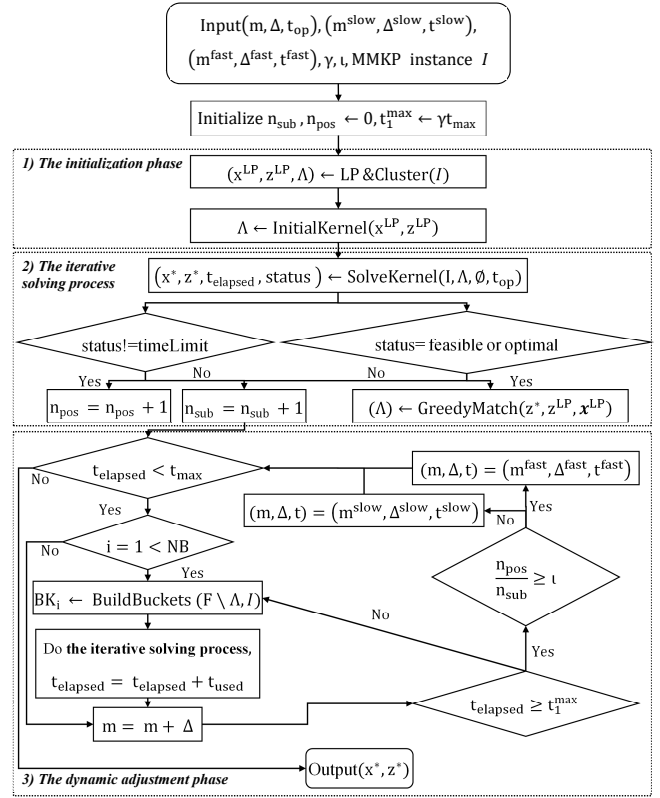


Fig. 2. Flowchart of the ACCO algorithm.

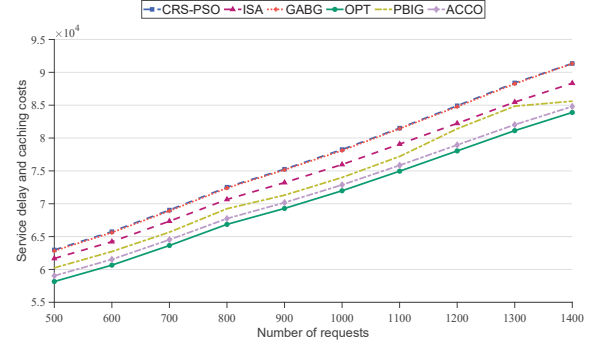


Fig. 3. Impact of the number of requests on service delay and caching costs. The ACCO algorithm provides near-optimal performance for solving the optimization problem, as the MMKP is NP-hard. However, the CRS-PSO, ISA, and GABG algorithms are prone to getting stuck in local optima. The PBIG algorithm's parameters are set randomly, and its greedy rules only consider the ratio of resource consumption to profit. In contrast, the ACCO algorithm solves subproblems iteratively using the CPLEX tool, which places contents with similar characteristics on the most appropriate edge node, taking into account optimization objectives and request frequency.

Fig. 4 details the effect of the Zipf parameter j on service latency and caching costs. A larger j means that user preferences are more focused on popular contents. However, the ACCO algorithm considers both service latency and caching costs, so the result is not monotonic. The weighted sum of service latency and caching costs increases steadily

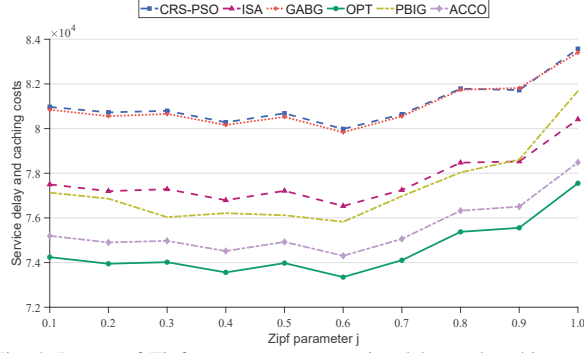


Fig. 4. Impact of Zipf parameter j on service delay and caching costs.

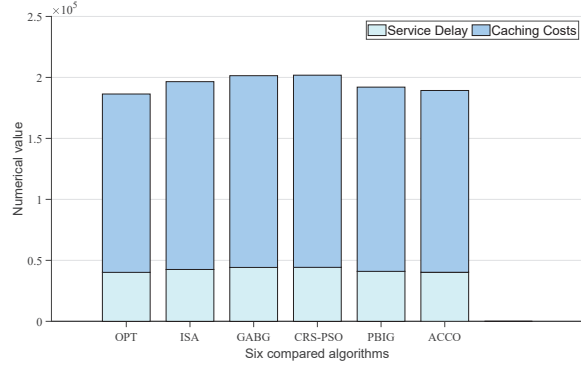


Fig. 5. The percentage of delay and costs of different algorithms.

when $j > 0.6$, and the cache efficiency of the ACCO algorithm is highest when $j = 0.6$. Changing j has the greatest impact on the PBIG algorithm.

In Fig. 5, the percentage of service delay and caching costs for each algorithm with 1000 requests is displayed. The six algorithms have similar caching costs due to similar contents sizes. Therefore, the BS only reduces service delay when the contents is large enough. The ACCO algorithm, which uses a k-means clustering algorithm and a greedy-based load balancing strategy, can greatly reduce caching costs and achieve the best cost performance between contents and edge nodes.

Furthermore, we compare the running time of each algorithm for a range of request numbers in Fig. 6. All six algorithms show an increase in running time as the number of requests increases. Although CPLEX provides the optimal solution, it requires a large computational overhead for large-scale scenarios. The ACCO algorithm, which implements the KS framework to reduce computational overhead, produces results that are closest to the optimal solution while maintaining low complexity. The PBIG algorithm reduces problem dimensions through the destruction and construction of partial solutions. Based on Figs. 3-6, we conclude that the ACCO algorithm is advantageous in different settings in terms of popularity, number of requests, and running time.

V. CONCLUSION

This paper investigates collaborative air-ground edge caching in dynamic vehicular networks, where UAVs, RSUs, and BS deliver popular contents to moving vehicles. We

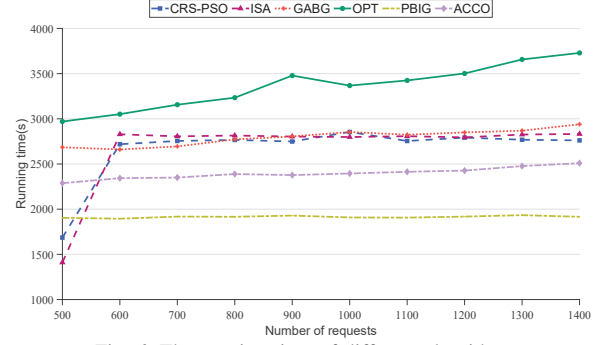


Fig. 6. The running time of different algorithms.

formulate the problem of heterogeneous cache placement as a generalized MMKP and propose the ACCO algorithm to solve this problem. The ACCO algorithm incorporates the k-means clustering algorithm and a greedy-based load balancing strategy to obtain the near-optimal cache matrix of UAVs and RSUs, improving continuous connectivity between moving vehicles and infrastructure. We demonstrate through performance evaluation that the proposed architecture and caching optimization algorithm can minimize service delay and caching costs for in-car entertainment applications.

REFERENCES

- [1] J. Chen, H. Wu, P. Yang, F. Lyu, and X. Shen, "Cooperative edge caching with location-based and popular contents for vehicular networks," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 9, pp. 10291–10305, Sept. 2020.
- [2] J. Yao, T. Han, and N. Ansari, "On mobile edge caching," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2525–2553, 2019.
- [3] H. Wu, J. Chen, W. Xu, N. Cheng, W. Shi, L. Wang, and X. Shen, "Delay-minimized edge caching in heterogeneous vehicular networks: A matching-based approach," *IEEE Transactions on Wireless Communications*, vol. 19, no. 10, pp. 6409–6424, Oct. 2020.
- [4] N. Zhao, F. Cheng, F. R. Yu, J. Tang, Y. Chen, G. Gui, and H. Sari, "Caching uav assisted secure transmission in hyper-dense networks based on interference alignment," *IEEE Transactions on Communications*, vol. 66, no. 5, pp. 2281–2294, May 2018.
- [5] M. Duan, "Management system project design of urban intelligent transportation," *Journal of Interdisciplinary Mathematics*, vol. 21, no. 5, pp. 1175–1179, 2018.
- [6] B. Jiang, J. Yang, H. Xu, H. Song, and G. Zheng, "Multimedia data throughput maximization in internet-of-things system based on optimization of cache-enabled uav," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 3525–3532, Apr. 2019.
- [7] L. Lamanna, R. Mansini, and R. Zanolini, "A two-phase kernel search variant for the multidimensional multiple-choice knapsack problem," *European Journal of Operational Research*, May 2021.
- [8] M. Feng and H. Pan, "A modified pso algorithm based on cache replacement algorithm," in *2014 Tenth International Conference on Computational Intelligence and Security*, Nov. 2014, pp. 558–562.
- [9] A. Liu, J. Wang, G. Han, S. Wang, and J. Wen, "Improved simulated annealing algorithm solving for 0/1 knapsack problem," in *Sixth International Conference on Intelligent Systems Design and Applications*, vol. 2, Oct. 2006, pp. 1159–1164.
- [10] J. Zhao, T. Huang, F. Pang, and Y. Liu, "Genetic algorithm based on greedy strategy in the 0-1 knapsack problem," in *2009 Third International Conference on Genetic and Evolutionary Computing*, Oct. 2009, pp. 105–107.
- [11] S. Bouamama and C. Blum, "On solving large-scale instances of the knapsack problem with setup by means of an iterated greedy algorithm," in *2017 6th International Conference on Systems and Control (ICSC)*, May 2017, pp. 342–347.